PyRpiCamController – CamController Architecture

This document provides a structured architectural overview of the CamController module. It is intended as developer documentation for understanding, maintaining, and extending the codebase.

Audience: Python developers, maintainers, and contributors

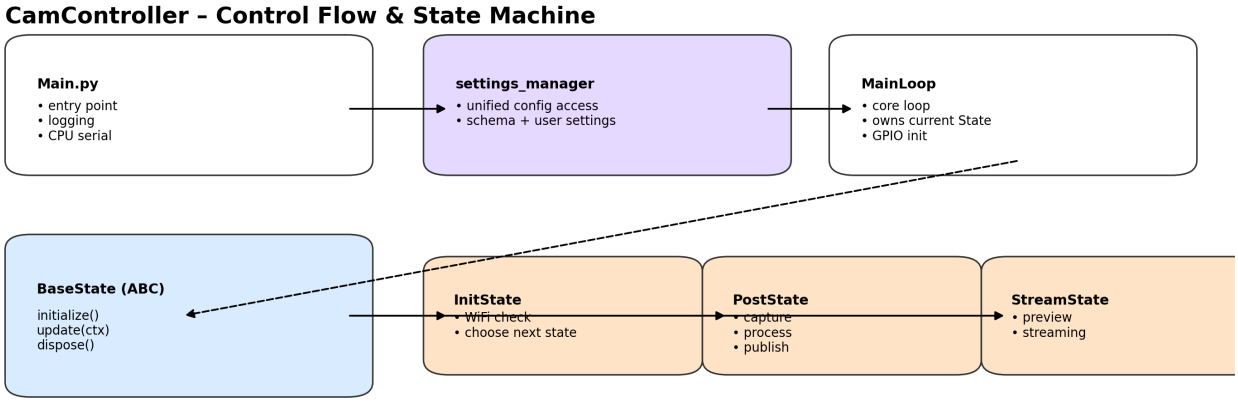
Scope: Control flow, state machine, camera abstraction, vision pipeline, publishers, and hardware integration

Table of Contents

1. Control Flow & State Machine
2. Camera Abstraction & Vision Pipeline
3. Publishers, Hardware & Streaming
4. Developer Recipes & Extension Guide

1. Control Flow & State Machine

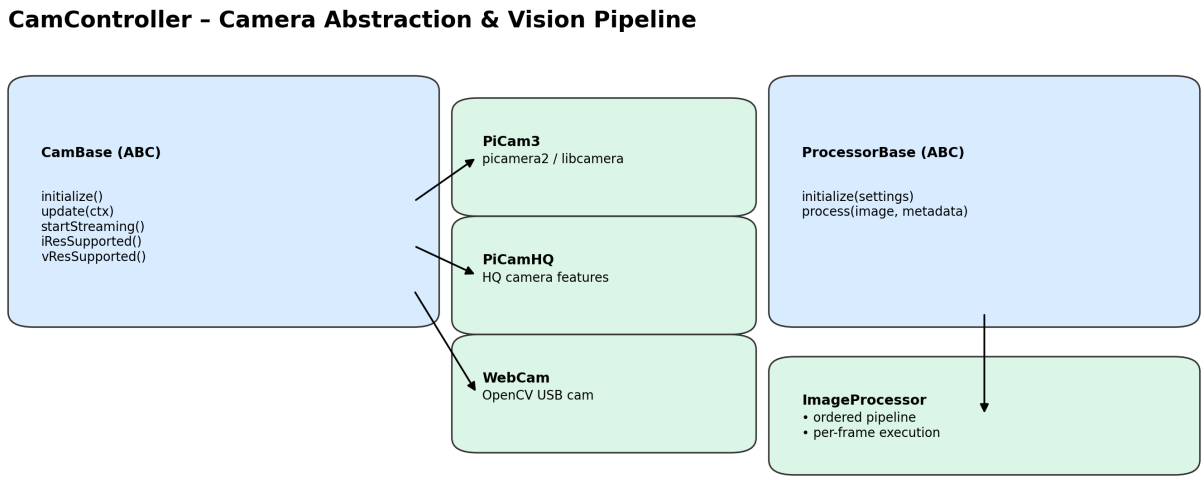
This section describes how execution starts in Main.py, how MainLoop manages the runtime, and how the State pattern is used to switch between Init, Capture/Publish, and Streaming modes.



Key concepts:
• MainLoop owns exactly one active State at a time
• settings_manager is the single source of configuration
• States decide transitions based on settings and environment

2. Camera Abstraction & Vision Pipeline

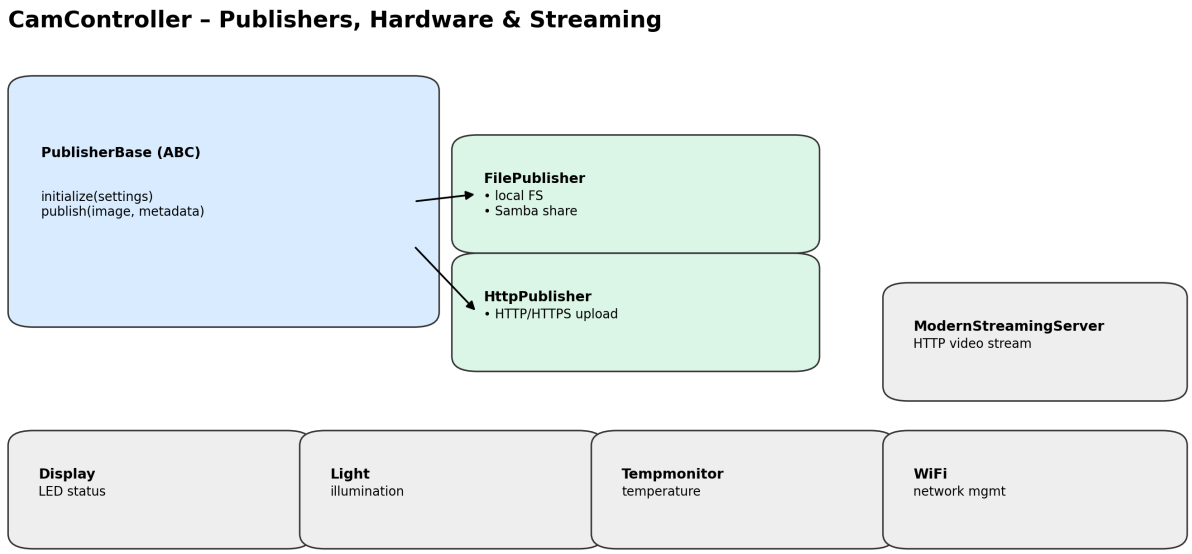
This section shows how camera hardware and image processing are decoupled using abstract base classes and composition.



Key concepts:
• All cameras inherit from CamBase
• ImageProcessor coordinates an ordered processor pipeline
• Cameras and processors can be extended independently

3. Publishers, Hardware & Streaming

This section documents how processed images are published, how hardware I/O is structured, and how live streaming is provided.



Key concepts:
• Publishing uses the Strategy pattern
• Hardware modules are thin and state-driven
• Streaming is isolated from capture/publish logic

4. Developer Recipes & Extension Guide

This section provides practical, step-by-step guidance for extending the CamController codebase in a safe and idiomatic way.

4.1 Adding a New Camera Type

1. Create a new module in Cam/ inheriting from CamBase
2. Implement required methods: initialize(), update(), startStreaming()
3. Declare supported resolutions
4. Register the camera in getCam(camtype)
5. Add configuration to the settings schema

4.2 Adding a New Vision Processor

1. Create a new processor inheriting from ProcessorBase
2. Implement process(image, metadata)
3. Validate inputs and return enriched metadata
4. Enable and configure it via the settings schema
5. ImageProcessor will automatically include it in the pipeline

4.3 Adding a New Publisher

1. Create a new publisher inheriting from PublisherBase
2. Implement initialize(settings) and publish()
3. Avoid blocking calls in publish()
4. Add enable flags and configuration under settings['Cam']['publishers']

4.4 Adding a New State

1. Create a new State inheriting from BaseState
2. Implement initialize(), update(), dispose()
3. Decide transitions explicitly in update()
4. Register state transitions in MainLoop if required

Design principle: Extend by inheritance and configuration, not by modifying existing states. This keeps the system modular and testable.